PROJET D'IHM 2026 · DOSSIER DE CONCEPTION

Les diagrammes du projet

Référentiel des types de diagrammes, choix argumenté
et diagrammes produits pour la plateforme Java · Python · C++

UML 2.5 | SysML · BPMN · C4 | IHM & wireframes | 7 diagrammes produits

Ce que contient ce dossier

A. Le catalogue complet des types de diagrammes (UML, SysML, BPMN, C4, Merise, gestion de projet, IHM)

B. Le choix argumenté pour ce projet : à produire, recommandé, inutile

C. Les 7 diagrammes du projet, en vectoriel

D. Conventions de notation, outillage et checklist de rendu

Les 7 diagrammes produits

1 · Cas d'utilisation — 2 · Classes — 3 · États-transitions — 4 · Séquence

5 · Activité — 6 · Composants — 7 · Déploiement

> **Règle de lecture :** un diagramme n'est jamais décoratif. Chacun répond à une question précise et se rattache à des exigences numérotées de l'analyse des besoins.

A · Catalogue des types de diagrammes

Deux grandes familles à ne pas confondre : les diagrammes **de structure** (ce que le système est) et les diagrammes **de comportement** (ce que le système fait). UML 2.5 en normalise 14 ; d'autres notations couvrent ce qu'UML ne traite pas (les exigences, le projet, l'IHM).

A1 UML 2.5 — diagrammes structurels (7)

Type	Question à laquelle il répond	Utile au projet ?	État
Classes	Quelles entités, quels attributs, quelles relations ?	**Indispensable**	✔ produit
Objets	Une photo du système à un instant donné	Non	—
Paquetages	Comment regrouper le code en modules ?	Recommandé	partiel (3 couches)
Composants	Quels blocs logiciels et quelles interfaces ?	**Indispensable**	✔ produit
Structure composite	Le détail interne d'un composant	Non	—
Déploiement	Sur quelles machines tourne quoi ?	**Indispensable**	✔ produit
Profil	Extension d'UML pour un domaine	Non	—

A2 UML 2.5 — diagrammes comportementaux (7)

Type	Question à laquelle il répond	Utile au projet ?	État
Cas d'utilisation	Qui fait quoi avec le système ?	**Indispensable**	✔ produit
Activité	Quel enchaînement d'actions et de décisions ?	**Indispensable**	✔ produit
États-transitions	Quels états successifs d'un objet ?	**Indispensable**	✔ produit
Séquence	Quels messages, dans quel ordre, entre qui ?	**Indispensable**	✔ produit
Communication	Même info que séquence, centrée sur les liens	Optionnel	—
Vue d'ensemble des interactions	Comment combiner plusieurs scénarios ?	Optionnel	—
Temps	Quelles contraintes temporelles ?	Optionnel	—

À retenir : dans un projet d'IHM, quatre diagrammes UML suffisent à porter 90 % de la conception — **cas d'utilisation** (le périmètre), **classes** (les données), **séquence** (le scénario critique) et **états-transitions** (la règle métier). Les autres sont du confort.

A3 · Autres notations et diagrammes utiles

Type / notation	À quoi ça sert	Verdict pour ce projet
SysML — Exigences	Relier exigences, justification et tests	**Fortement recommandé** (projet d'ingénierie)
SysML — Blocs / paramétrique	Systèmes physiques, contraintes d'ingénierie	Non pertinent ici
BPMN 2.0	Processus métier entre plusieurs organisations	Optionnel (variante de l'activité)
C4 model (contexte, conteneurs)	Architecture lisible par des non-techniciens	Recommandé — très efficace en soutenance
ArchiMate	Urbanisation du SI, alignement métier/technique	Non pertinent
MCD / MLD (Merise)	Modèle conceptuel puis logique des données	Recommandé (équivalent du diagramme de classes)
Modèle entité-association (ER)	Variante anglo-saxonne du MCD	Optionnel (redondant avec classes)
DFD (flux de données)	Circulation des données entre traitements	Optionnel
Diagramme de Gantt	Planification des tâches dans le temps	**Recommandé** pour le planning du projet
PERT / réseau d'antécédents	Dépendances et chemin critique	Optionnel
WBS / organigramme des tâches	Découpage du travail	Recommandé (répartition d'équipe)
Organigramme	Structure hiérarchique d'une organisation	Non pertinent
Carte mentale (mind map)	Brainstorming, arborescence d'idées	Utile en atelier
Ishikawa (arêtes de poisson)	Analyse des causes d'un problème	Optionnel (risques)
Matrice de traçabilité	Exigence → conception → test	**Indispensable** (souvent demandée)
User journey / empathy map	Vécu de l'utilisateur, points de friction	**Fortement recommandé** en IHM
Story map	Découpage du produit par étapes d'usage	Recommandé
Wireframe / zoning	Structure d'un écran, sans graphisme	**Indispensable** en projet d'IHM
Maquette haute fidélité	Rendu final de l'écran	**Indispensable**
Prototype interactif	Démonstration cliquable du parcours	Recommandé
Charte graphique / design system	Polices, couleurs, composants	Recommandé

B · Ce qu'il faut produire pour ce projet

B1 Priorité 1 — indispensables (à joindre au rendu)

Diagramme	Ce qu'il prouve au correcteur	Exigences couvertes
Cas d'utilisation	Que le périmètre est compris : 3 acteurs, 6 modules, le moteur de validation	UC-01 → UC-20
Classes	Que les données sont modélisées, y compris **un parcours par langage**	BF-*, RG-20 → RG-24
États-transitions	Que la règle de progression est **déterministe** et non « au feeling »	BF-N1 → BF-N8, RG-04/RG-06
Séquence	Que le scénario critique (passer un test) est maîtrisé de bout en bout	UC-08, UC-09, BF-E8, BNF-L
Activité	Que le processus pédagogique complet est compris	§5 du sujet
Composants	Qu'une architecture est proposée, avec des responsabilités séparées	BNF-09, BNF-IA15
Déploiement	Que le risque d'exécution de code (3 langages) est anticipé	BNF-01, BNF-L1 → L6
Wireframes des écrans clés	Que l'ergonomie a été conçue, pas subie (c'est un projet d'IHM)	IHM-01 → IHM-28
Matrice de traçabilité	Qu'aucune exigence n'est oubliée ni inventée	§9 de l'analyse

B2 Priorité 2 — recommandés (font la différence en soutenance)

Diagramme	Pourquoi il vaut le coup ici
Diagramme d'exigences (SysML)	Visualise les 148 exigences et leurs liens `«derive»` / `«satisfy»` — rare et très apprécié
C4 niveau 1 et 2	Explique l'architecture en 2 images à un jury non technique
User journey de l'apprenant	Montre les points de friction : niveau verrouillé, échec, reprise
Wireframes + maquette haute fidélité	Le cœur de l'évaluation d'un projet d'IHM
Gantt + WBS	Prouve que le projet (3 langages !) est planifié et réparti
Diagramme de flux de code	Explique la chaîne soumission → sandbox → note, très technique

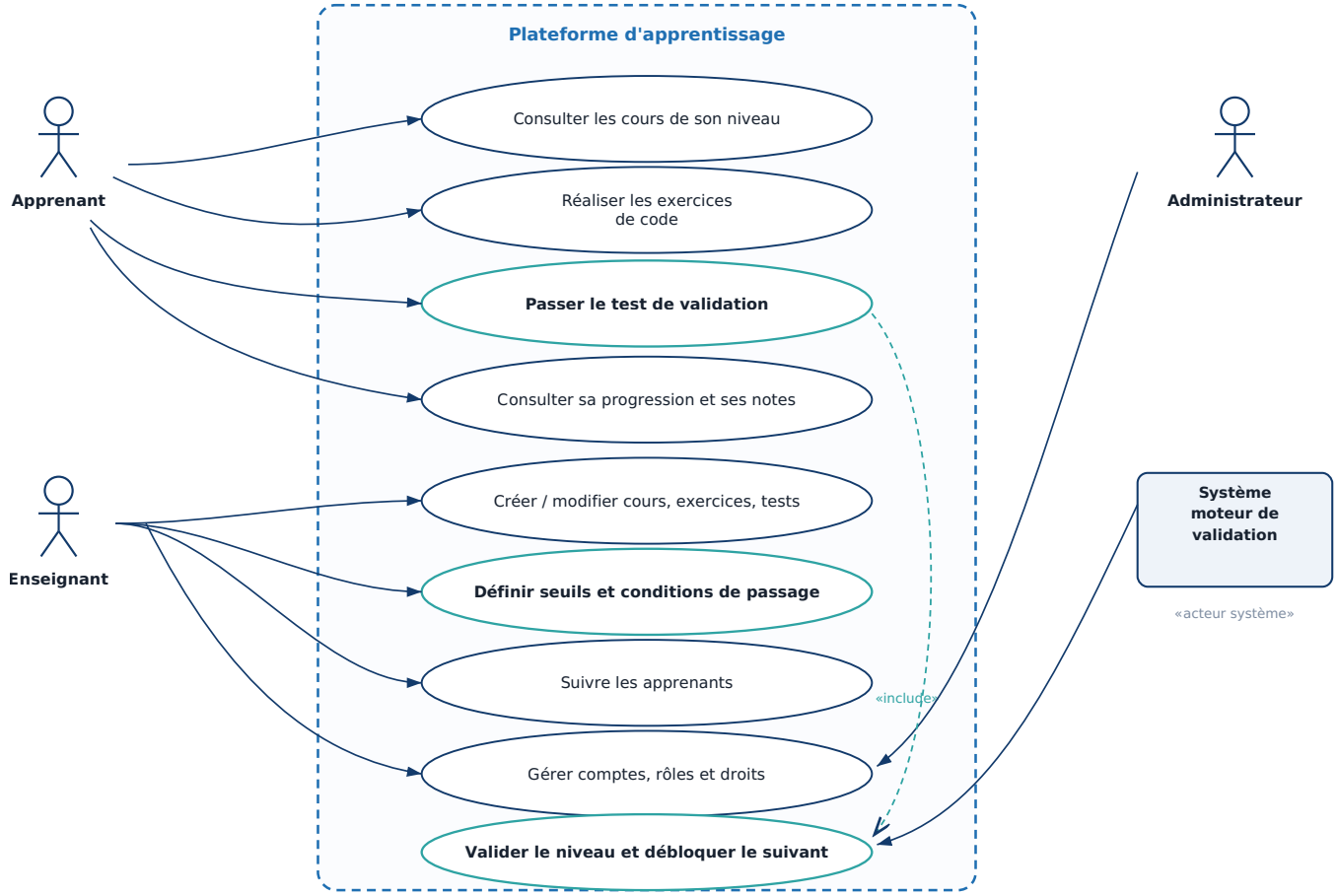
B3 À écarter — inutiles ici

Diagramme d'objets, structure composite, profil UML, diagramme de temps, ArchiMate, organigramme, PERT complet, diagramme de communication en doublon avec la séquence. **Chaque diagramme produit doit être défendu à l'oral** : un diagramme que l'on ne sait pas expliquer coûte plus qu'il ne rapporte.

C · Les 7 diagrammes du projet

1 Cas d'utilisation

Diagramme de cas d'utilisation — plateforme Java · Python · C++

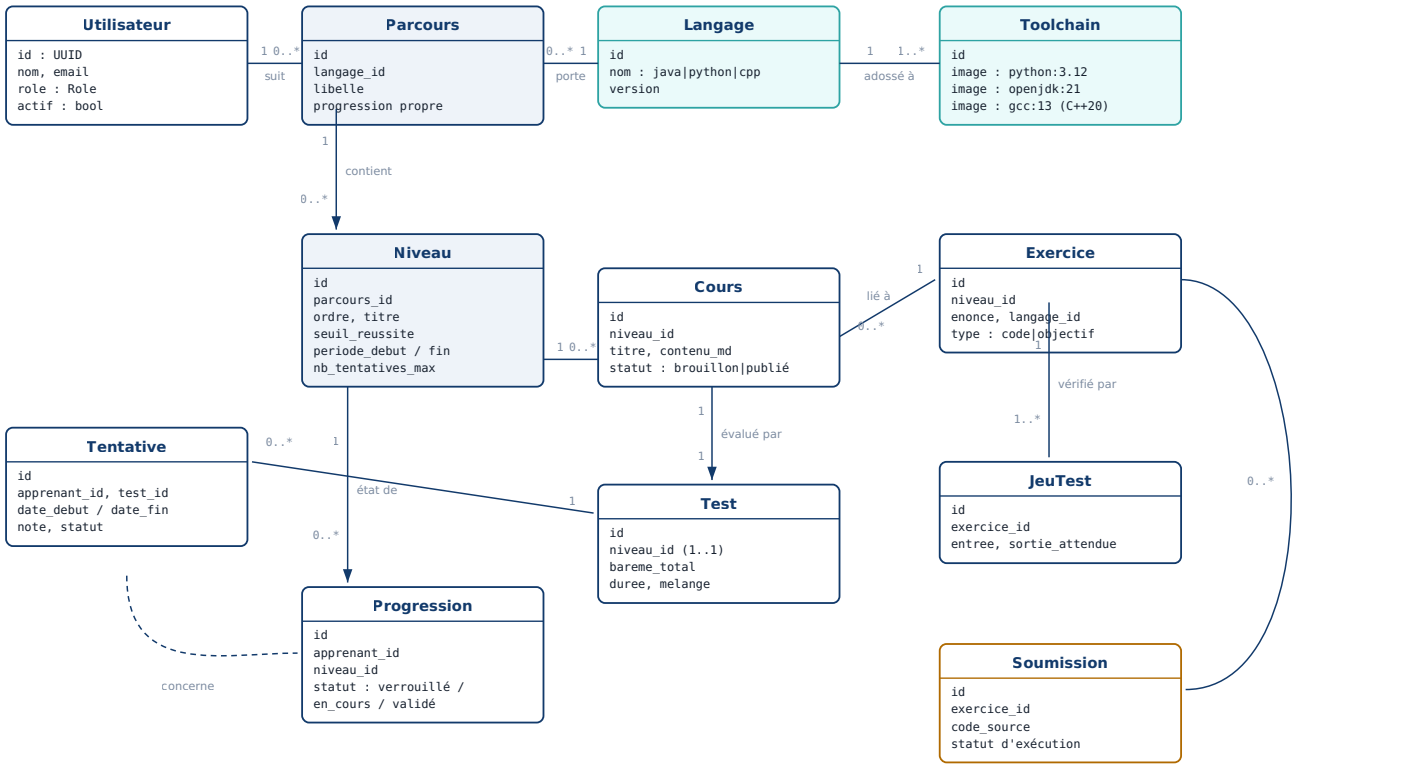


Les cas en gras portent le cœur du sujet : la validation conditionne le déblocage.

Ce qu'il faut dire à l'oral : trois acteurs humains, un acteur système (le moteur de validation) — c'est lui qui porte la valeur du sujet, car il décide du déblocage.

2 Classes — modèle de données

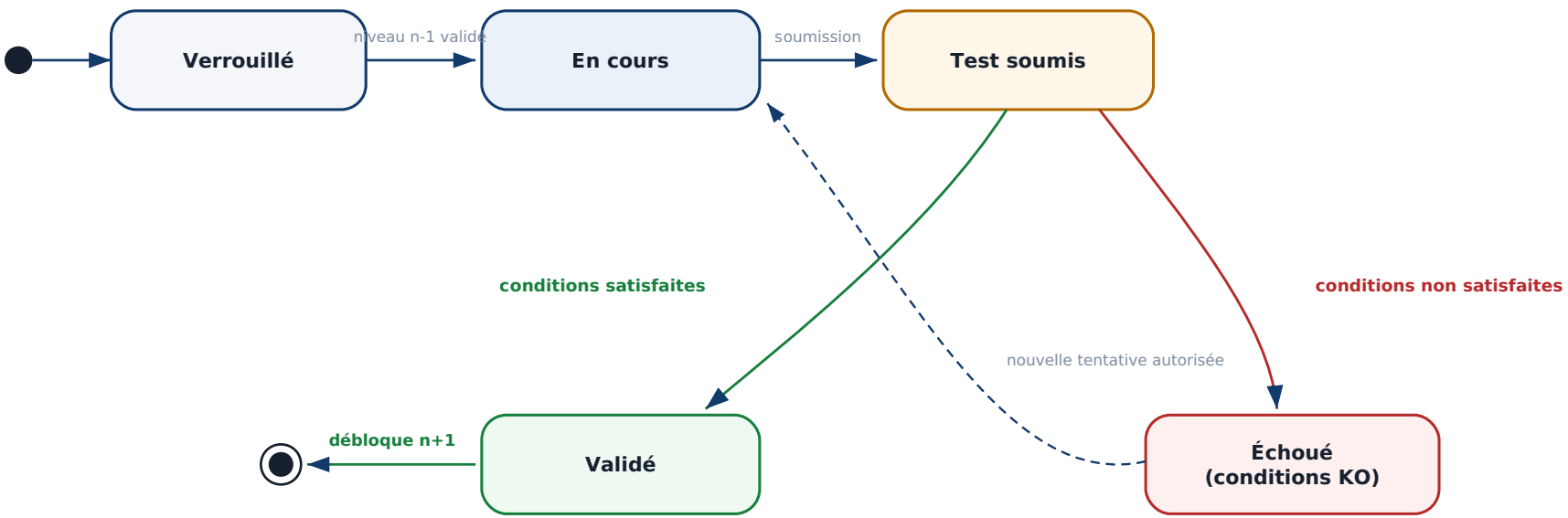
Diagramme de classes — modèle de données (3 langages)



Point clé : l'entité `Langage` et le couple `Parcours → Niveau` rendent les trois langages paramétrables : **le langage est une donnée, pas du code**. C'est ce qui évite de tripler l'application.

3 États-transitions — cycle de vie d'un niveau

Diagramme d'états-transitions — cycle de vie d'un niveau

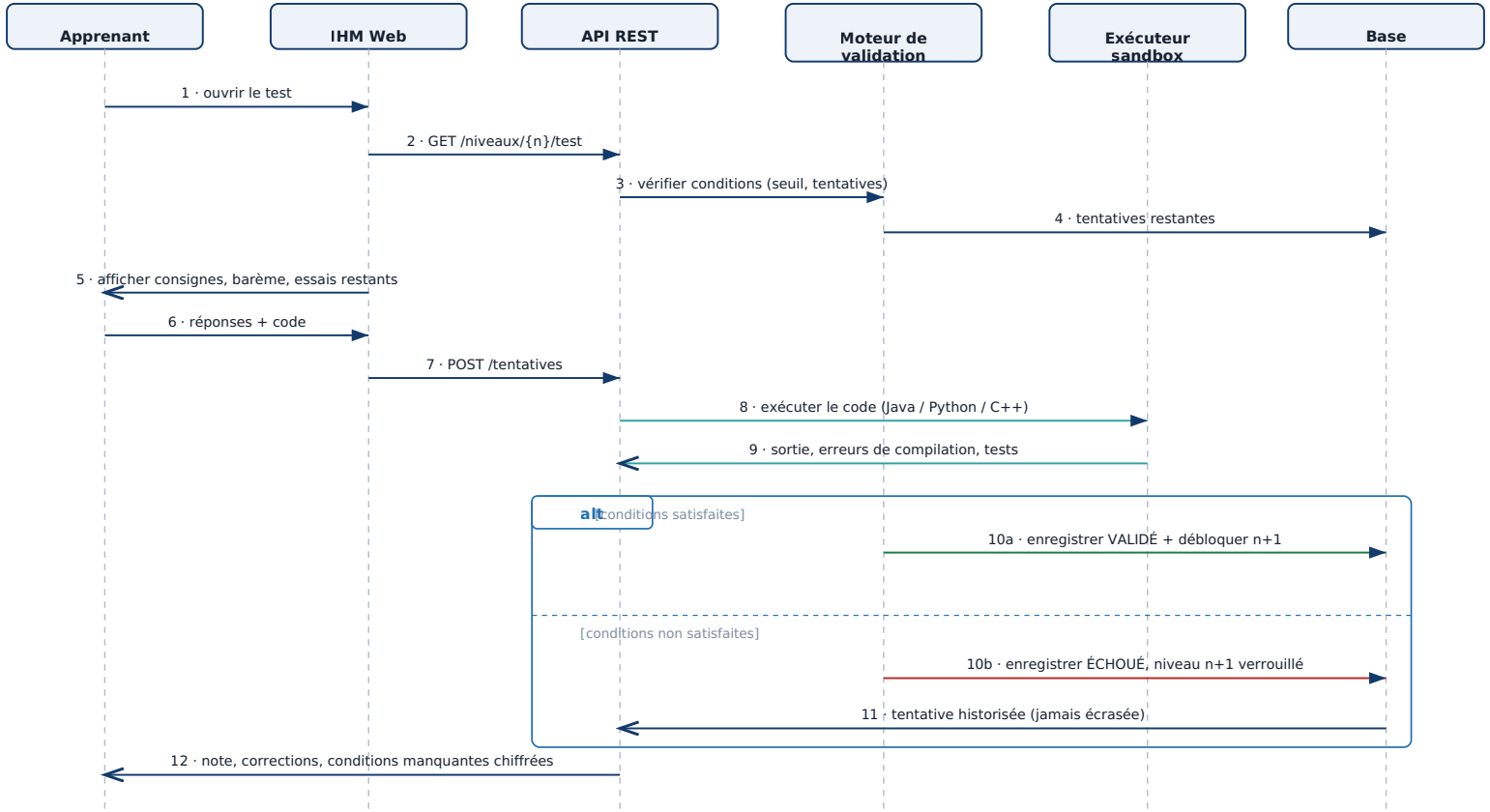


Un niveau validé ne redevient jamais verrouillé (sauf décision explicite de l'enseignant, Q5).
Chaque tentative est historisée ; l'état de progression est recalculable depuis les tentatives.

Ce qu'il démontre : la progression est un **automate déterministe**. Un niveau validé ne redevient jamais verrouillé, et l'échec n'est pas une impasse : il autorise une reprise.

4 Séquence — « passer le test de validation »

Diagramme de séquence — « passer le test de validation » (UC-08 / UC-09)

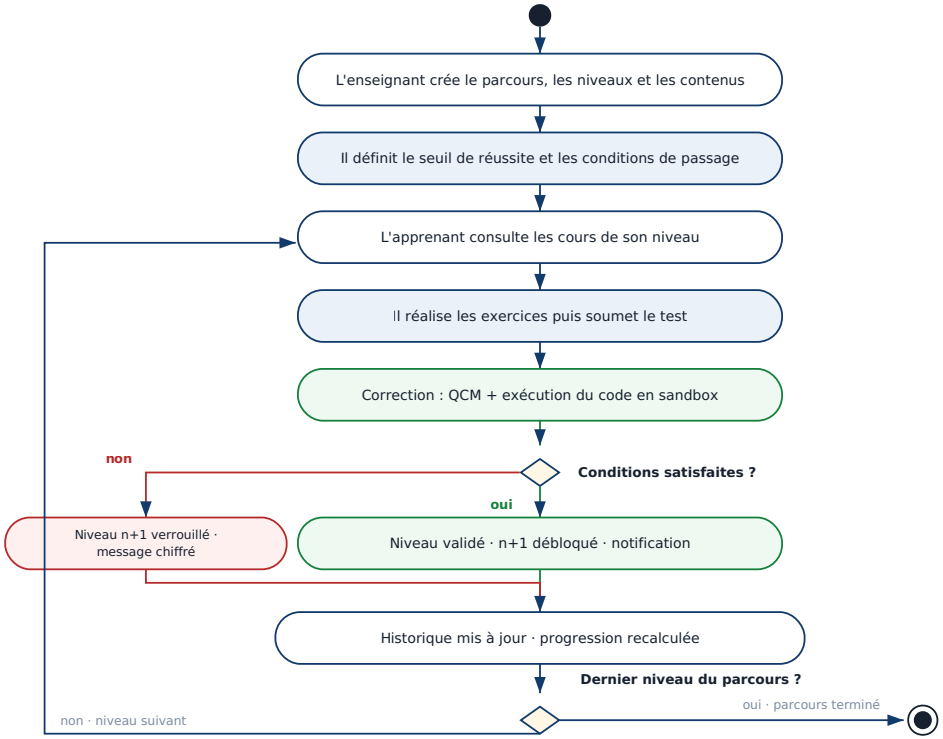


La décision est déterministe : le moteur compare la note aux conditions de l'enseignant. L'agent IA n'intervient jamais dans ce scénario (RG-13).

Le scénario critique du sujet. On y voit l'exécution du code déléguée à un exécuteur isolé, puis la décision prise par le moteur seul — l'agent IA n'apparaît jamais dans ce flux.

5 Activité — processus de progression

Diagramme d'activité — processus de progression

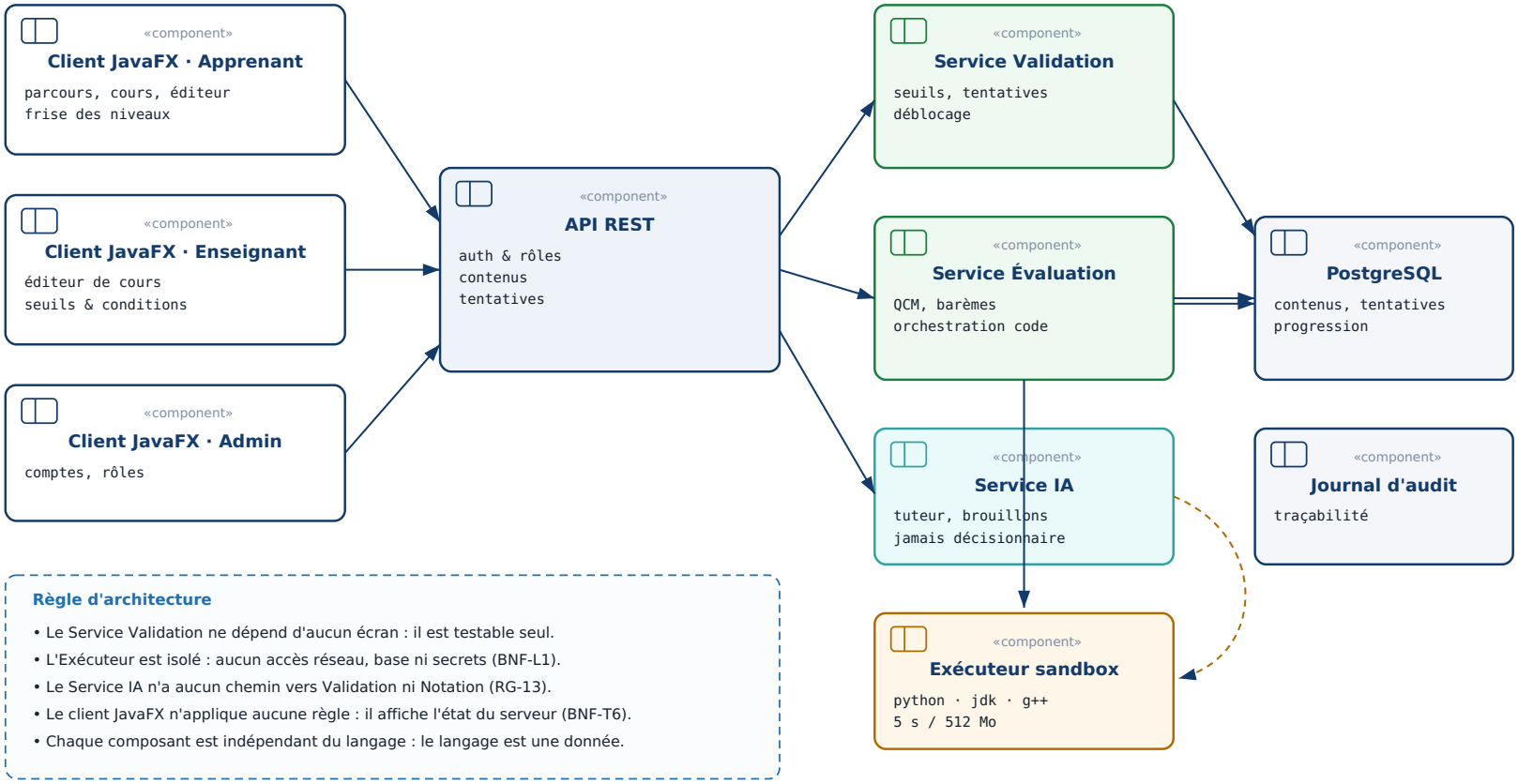


Le chemin « non » renvoie l'apprenant sur le niveau suivant, sans jamais franchir un niveau non validé.

Vue « métier » du §5 du sujet : huit étapes, deux issues, une boucle de reprise. C'est le diagramme le plus lisible pour un enseignant non informaticien.

6 Composants — architecture applicative

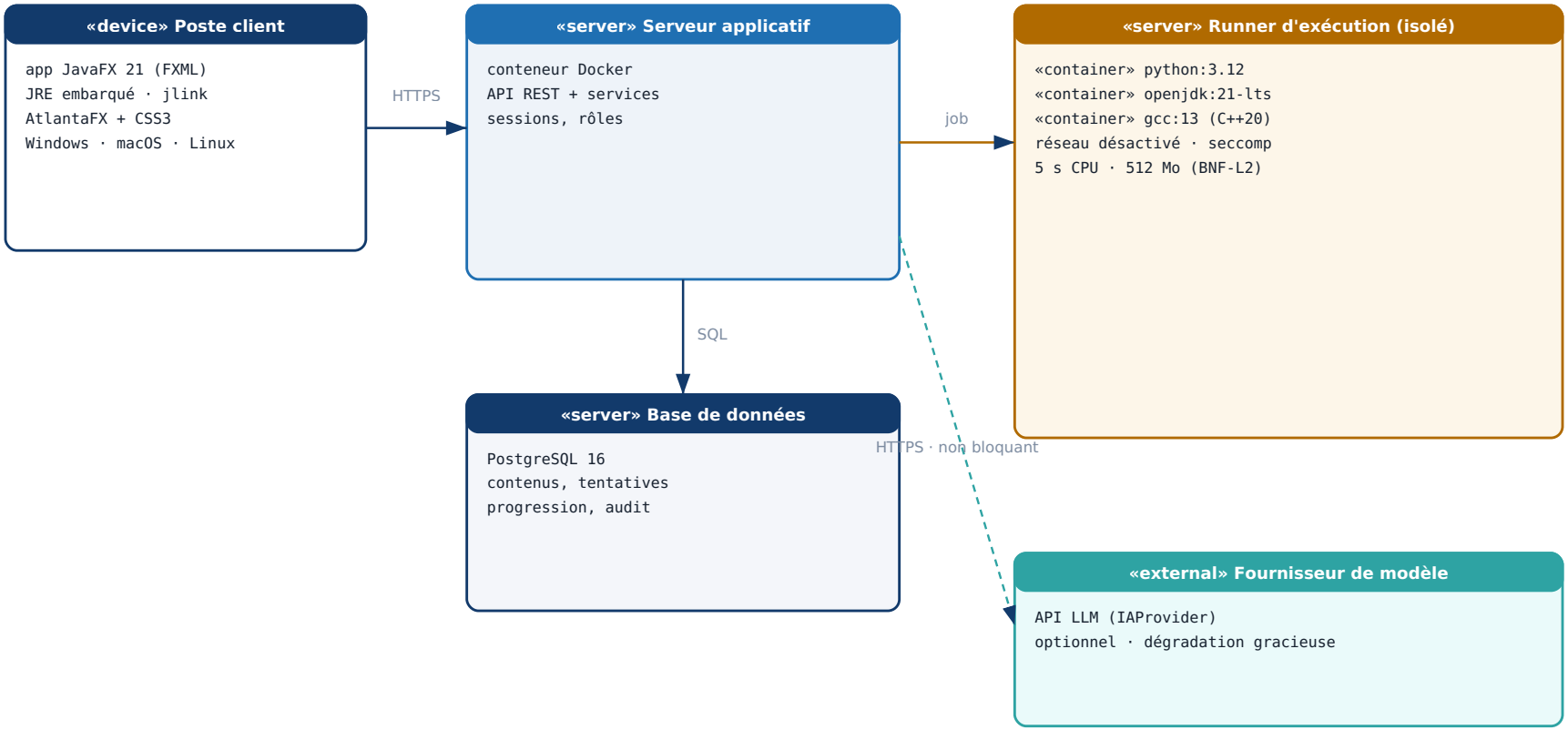
Diagramme de composants — architecture applicative



Règle d'architecture mise en avant : le service de validation ne dépend d'aucun écran, et le service IA n'a aucun chemin vers la notation (RG-13).

7 Déploiement — hébergement et isolation

Diagramme de déploiement — hébergement et isolation



Client JavaFX (bureau) + serveur applicatif : l'exécution du code apprenant reste le seul point à risque — conteneurs éphémères, sans réseau, avec quotas (BNF-L1 → L7).

Le vrai risque technique du projet : exécuter du code étudiant en Java, Python et C++ sans compromettre le serveur. Réponse : conteneurs éphémères, sans réseau, avec quotas.

D · Conventions, outillage et checklist

D1 Conventions de notation à respecter

Point	Règle appliquée dans ce dossier
Nommage	Acteurs au singulier, cas d'utilisation à l'infinitif (Passer le test de validation), classes au singulier en français
Multiplicités	Toujours les deux côtés : `1` · `0..*` · `1..*`
Encadré système	Une frontière explicite par diagramme de cas d'utilisation
Flèches	Ouvertes pour les messages et dépendances, triangle plein pour l'héritage, losange plein pour la composition
Scénarios	Un fragment `alt` par alternative majeure dans les séquences
Légende	Chaque diagramme porte une phrase de lecture : ce qu'il faut en retenir
Traçabilité	Chaque diagramme cite les identifiants d'exigences qu'il couvre (BF-, RG-, IHM-)

D2 Outils

Outil	Pour quoi	Remarque
draw.io / diagrams.net	Tous les diagrammes, export SVG et PDF	Le plus simple, gratuit, hors ligne
PlantUML	Diagrammes écrits en texte, versionnables	Idéal en équipe : le diagramme suit le code
Mermaid	Séquences et états dans le Markdown	Rendu direct dans GitLab/GitHub
StarUML / UMLet	UML rigoureux, conforme au méta-modèle	Utile si le correcteur est pointilleux
Figma	Wireframes et maquettes haute fidélité	Indispensable pour la partie IHM
Ce dossier	SVG générés, vectoriels et modifiables	Sources dans `diagrammes/`

D3 Checklist de rendu

Avant de rendre, vérifier :
① chaque diagramme a un **titre** et une **phrase de lecture** · ② toute flèche a une **direction justifiable** · ③ les **multiplicités** sont présentes des deux côtés · ④ le **vocabulaire** est identique à celui de l'analyse des besoins · ⑤ chaque diagramme est cité **dans le texte** du rapport (« cf. figure 4 ») · ⑥ le format est **vectoriel** (SVG ou PDF), jamais une capture d'écran floue · ⑦ les **exigences** couvertes sont indiquées sous chaque figure.

Piège classique : produire douze diagrammes et n'en maîtriser aucun. Mieux vaut **sept diagrammes justes**, cohérents avec l'analyse des besoins, que vingt diagrammes approximatifs dont trois se contredisent.